

CHAPTER 5

Enterprise Metamodel



This chapter is part of the GREEN, BLUE, and GOLD reading tours. See Figure 1-1 in Chapter 1, Section 1.3, for alternative reading tours.

Every entity used in architecture work and described in the (Extended) Content Framework is rooted in the Enterprise Metamodel. This makes the Enterprise Metamodel a fundamental part of the TOGAF Standard.

Since no two organizations are alike, the Enterprise Metamodel serves as a Foundation Enterprise Metamodel. This means that it must be tailored to the needs of the organization. The metamodel described in the TOGAF Framework is best viewed and used as a starting point.

The Content Framework provides a categorization mechanism that can be used to structure a representation of the Enterprise Metamodel, consisting of five interrelated sections. It is not to be confused with an architecture diagram that shows domains or layers of the architecture.

5.1. Sections of the Enterprise Metamodel

One of the core components of the TOGAF Standard is the Enterprise Metamodel. This metamodel contains the most common entities and their relationships as they are likely to occur in practice in most organizations. The Enterprise Metamodel *provides the context* for the specific artifacts

referenced in the descriptions of the Architecture Development Method phases (see Chapter 12). The Content Framework described in Chapter 6 *defines the types of architecture work products* and their relationships, based on the *entities* from the Enterprise Metamodel. Both the Enterprise Metamodel and the Content Framework are kept in sync; the Enterprise Metamodel provides the entities used in the Content Framework, and the Content Framework evolves the Enterprise Metamodel.

The Enterprise Metamodel comprises five interconnected *sections*, which correspond to the four architecture domains (Business, Data, Application, and Technology) and one generic section. As the name suggests, the Enterprise Metamodel is a metamodel, not an architecture diagram. Therefore, the term *sections* is used.

The five interrelated sections of the Enterprise Metamodel are:

- General Entities
- Business Architecture
- Data Architecture
- Application Architecture
- Technology Architecture

To improve readability, the relations between entities have been omitted from Figures 5-1, 5-2, 5-5, 5-14, 5-17, and 5-19.

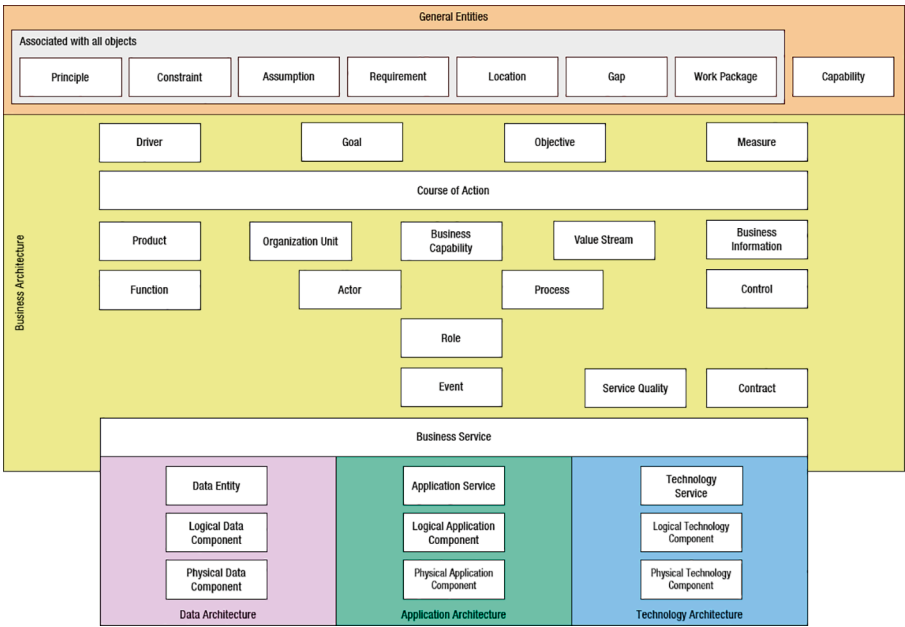


Figure 5-1. *The Foundation Enterprise Metamodel*

Each section consists of a set of model entities [11]. These entities are architecture concepts, each with its own definition and explanation. Understanding what the various entities stand for, what they mean, and how they can and should be used is critical to understanding the architecture products described in the TOGAF Framework. Having a clear definition ensures that there is no misunderstanding about how to use the concepts. For the above reasons, the entities are described in more detail in the following paragraphs.

5.1.1. General Entities

The upper section of the Enterprise Metamodel consists of what is called *General Entities* in the TOGAF Standard. It contains eight entities.

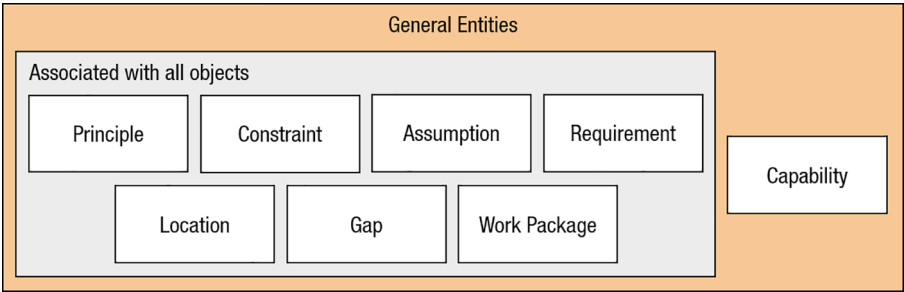


Figure 5-2. *General Entities of the Enterprise Metamodel*

These entities are concepts that are so generic in nature that they cannot be captured or categorized in any of the four core architecture domains. They can be used across all four domains. Table 5-1 provides an overview of the architecture concepts (entities) that are classified as General Entities.

Table 5-1. *General Entities*

Entity	Description
Principle	A qualitative statement of intent that should be met by the architecture. Aside from the statement, it has at least a descriptive name, supporting rationale, and one or more implications.
Constraint	An external factor that prevents an organization from pursuing particular approaches to meet its goals.
Requirement	A quantitative statement of business need that must be met by a particular architecture or work package.
Assumption	A statement of probable fact that has not been fully validated at this stage, due to external constraints.
Location	A place where activities occur. Locations can be composed and decomposed.

(continued)

Table 5-1. (continued)

Entity	Description
Gap	A statement of difference between two states. Used in the context of gap analysis, where the difference between the Baseline and Target Architecture is identified.
Work package	A set of actions identified to achieve one or more objectives for the business. A work package can be a part of a project, a complete project, or a program.
Capability	An ability that an organization, person, or system possesses.

As mentioned earlier, the architecture entities from the upper section of the Enterprise Metamodel can be used or related in the context of any architecture domain. For example, the *principle* entity can be related to an element from the Business Architecture domain or from the Technology Architecture domain. The *location* entity can be related to either a (logical or physical) application component or a role or actor. Due to their ability to be related to all architecture domains and their underlying elements, the TOGAF Standard refers to these entities as General Entities.

In this book, I will explain certain topics, concepts, and methods by providing examples. These examples are based on a fictional healthcare organization for which an architecture has been developed.

All examples in this and subsequent chapters relate to this organization and its situation. This approach ensures consistency throughout the book.

Principle: A qualitative statement of intent that should be met by the architecture. It has at least a supporting rationale and a measure of importance.

Principles are used to provide a high-level framework for architecture initiatives. There are Architecture Principles and Enterprise Principles (see Chapter 14, Section 14.2). *Principles* are similar to the constitution of a country. They have a high level of abstraction and serve as a kind of guiding light that can be used to give further direction and interpretation to their application and use. Principles are highly influential and strategic in nature and carefully designed to fit the organization and its business model. Like a constitution, these principles change little or nothing and have a strong binding character. The high-level or basic principles are based on business goals and objectives [12]. An example of the entity *principle* is provided in Figure 5-3.

Constraint: A factor – mostly external – that prevents an organization from pursuing particular approaches to meet its goals and objectives.

Stakeholders often bring constraints along with their concerns. For example, there may be a desire or necessity within the organization to create a new function (business role). However, if contractual arrangements have been made with an external party that provides staff to the organization, and that party uses standard job profiles, then the new role may not fit within the applicable standard. This restriction – *constraint* – makes it impossible to create the desired position.

Requirement: A *requirement* indicates the changes needed to a new or existing situation, environment, process, or system within the context of an initiative (work package). Requirements further specify the objective to be achieved and therefore represent a statement of a business need that must be met by a particular architecture or work package. They represent the essential attributes or behaviors that an enterprise system, architecture, or solution must have to achieve the organization's goals, objectives, and business outcomes. Requirements are used to guide the design, development, and validation of the architecture.

The relationship between a principle, a requirement, and a constraint is shown in Figure 5-3.

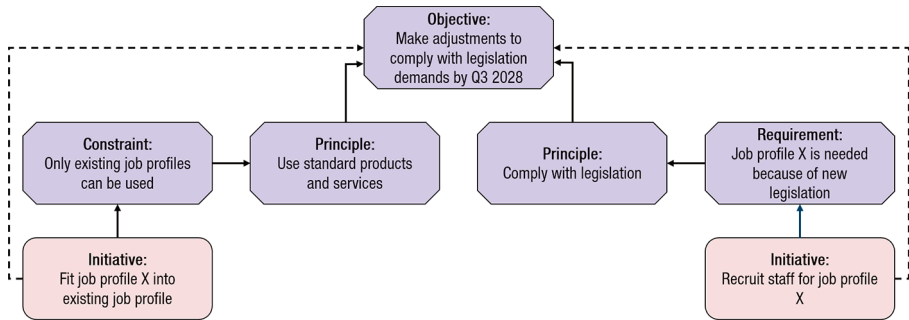


Figure 5-3. Relationship between a principle, requirement, and constraint

Figure 5-3 shows two principles. The principle on the left has a relationship with a constraint, and the principle on the right has a relationship with a requirement. The constraint has a relationship with an initiative (work package), and the requirement has a relationship with another initiative. Both initiatives affect the achievement of the objective, and both principles are also related to it. The situation described in Figure 5-3 demonstrates that the objective has been given a framework in the form of two principles. The principle on the right can be further detailed by the requirement and implemented by the associated initiative. The principle on the left is limited in its application by the constraint. However, an initiative is linked to the constraint in the hope of removing it.

Assumption: A statement of probable fact that has not been fully validated at this stage, due to external constraints.

An *assumption* is a foundational element used in architectural practices to define and communicate uncertainties or gaps in information that could influence decisions during the architecture development process. Assumptions are necessary when there is incomplete knowledge or uncertainty about factors impacting the architecture, but decisions still need to be made in order to proceed.

For instance, it may be assumed that an existing application will support a certain set of functional requirements, although those requirements may not yet have been individually validated. Assumptions are commonly used in project management. The TOGAF Series Guide “Architecture Project Management” elaborates on the added value of formulating assumptions.

Location: A place where activity happens. Literally. This entity refers to actual (physical) locations, for example, the geographic location of the office building the organization occupies or the countries or continents in which an organization operates, to name a few. The entity *location* can also be used in relation to information systems, applications, and technology components. A common diagram, often used by IT Architects, is that of the distribution of systems (think of server systems) across different data centers. In this diagram, the entity *location* is used to denote the different data centers.

Gap: This entity represents the difference between two states of architecture. It is most commonly used in the context of performing a gap analysis, where the difference between the Baseline and Target Architecture is identified.

A *gap* refers to a specific difference or shortfall between two states of an Enterprise Architecture: the *Baseline Architecture* (current state) and the *Target Architecture* (desired future state¹). Identifying and analyzing these gaps helps organizations understand what is missing, incomplete, or needs to change to move from the baseline to the target. A gap details the results of a gap analysis in phases B, C, and D of the Architecture Development Methodology and is an important input for subsequent implementation and migration planning.

¹ The *desired future state* should not be considered definitive in any way. Rather, it refers to a state at a specific point in time.

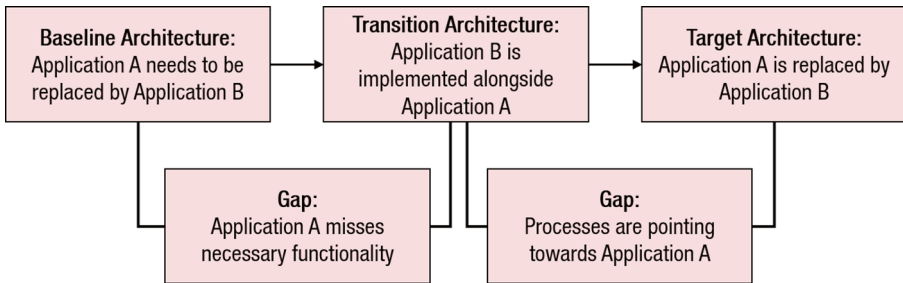


Figure 5-4. *Gaps between Baseline, Transition, and Target Architectures*

Continuing with the example shown in Figure 5-3, an application may be required to support certain functionality in order to execute the initiatives illustrated in the figure. This functionality is currently missing from the existing application (Application A). In order to achieve the objective shown in Figure 5-3, it has been decided to replace Application A with Application B. However, this will not happen overnight, so a phased approach has been chosen. First, Application B is implemented alongside Application A, and then all data and interfaces are transferred to Application B. Finally, Application A is decommissioned. This scenario is illustrated in Figure 5-4 using the *gap* entity.

Work Package: A set of actions identified to achieve one or more objectives for the business. A *work package* can be a part of a project, a complete project, or a program. It is a key concept used to manage and structure the implementation of an architecture. It represents a defined set of activities designed to achieve specific business objectives and outcomes as part of the overall organizational transformation.

Another word for work package is *initiative*. An initiative is a concept used to represent a planned action or project designed to achieve specific goals or objectives within the organization. Initiatives are concrete activities that are undertaken to implement changes, improve processes, or achieve desired outcomes as part of the overall strategy of the organization. They are directly linked to specific goals or objectives

of the organization (see Figure 5-3). Initiatives represent the actionable steps taken (projects, programs, or activities that implement architectural changes) to achieve these goals.

Work packages are also used to represent the activities required to close the gaps as you move from a Baseline Architecture through a Transition Architecture to a Target Architecture.

Capability: The term *capability* refers to an ability that an organization, system, or individual must possess. It represents a key construct in the TOGAF Framework, central to *capability-based planning*, which is an approach to Enterprise Architecture focusing on what an organization is able to do, rather than simply what it has.

A capability is an entity that focuses on what an organization (or a person, or even an object) is capable of doing. For example, consider an airplane. It has the ability to transport cargo or people from point A to point B. Of course, this capability is not unique to an airplane, because a car has the same capability. What makes an airplane different from other forms of transportation is that it has the ability *to fly*.

Another example of a capability, in this case of a motorcycle, is that it has the ability to *maneuver between cars* (think of traffic jams). Even though the motorcycle does not have a button that can be pressed to activate this ability, it still *has* this ability. Unlike an airplane that can actually fly on autopilot, the motorcycle would need its rider to perform the capability.

Capabilities define what an organization, person, or object should be able to do. The ability to actually execute the capabilities depends on having people, processes, applications, and/or technology available to perform the tasks necessary to execute the abilities. In architectural terms, a capability defines what an organization can (or should) do. It would then require actors, organizational units, processes, applications, and even technology to execute it. A capability therefore defines, at a conceptual level, what abilities an organization must have in order to do what it does (or wants to do).

The capability entity stands out from the rest of the General Entities because in addition to this entity, the Enterprise Metamodel also includes the *business capability* entity (see Section 5.1.2). However, these two types of entities are different from each other. The business capability entity has the following definition:

A business capability is a particular ability or capacity that a business may possess or exchange to achieve a specific purpose or outcome [13].

A business capability thus differs from a capability in that the former has an *actionable component* in its definition. The bit about *to achieve a specific purpose or outcome* indicates that it must indeed have a specific purpose. The regular capability does not have this addition in its definition. This makes business capabilities more meaningful compared with regular capabilities. After all, simply having an *ability* says nothing about its usefulness and necessity. By adding a clear purpose (*to achieve a specific purpose or outcome*) to a regular capability, it suddenly becomes valuable.



Further Reading

The TOGAF Series Guide “Architecture Project Management”, document number G188, provides additional information on applying the TOGAF Standard to project management.

5.1.2. Business Architecture

The most comprehensive section of the Enterprise Metamodel consists of Business Architecture Entities. Nineteen in total.

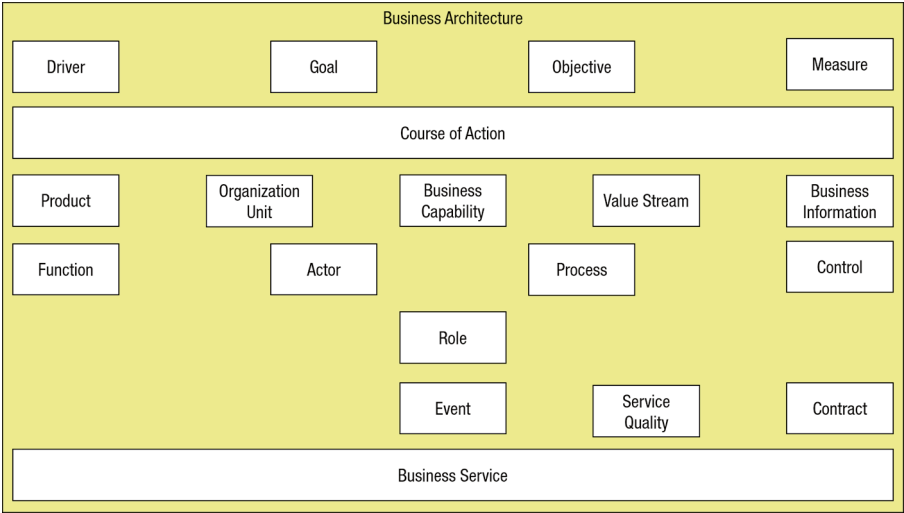


Figure 5-5. *Business Architecture Entities of the Enterprise Metamodel*

Table 5-2 lists the architecture concepts (entities) that can be classified as Business Architecture Entities.

Table 5-2. *Business Architecture Entities*

Entity	Description
Driver	An external or internal condition that motivates the organization to define its goals.
Goal	A high-level statement of intent or direction for an organization. Typically used to measure success of an organization.

(continued)

Table 5-2. *(continued)*

Entity	Description
Objective	An organizational aim that is declared in a Specific, Measurable, Achievable, Realistic, and Timebound (SMART) way.
Measure	An indicator or factor that can be tracked, usually on an ongoing basis, to determine success or alignment with objectives and goals.
Course of action	Direction and focus provided by strategic goals and objectives, often to deliver the value proposition characterized in the business model.
Business capability	A particular ability that a business may possess or exchange to achieve a particular purpose.
Value stream	A representation of an end-to-end collection of activities that create an overall result for a customer, stakeholder, or end user.
Function	A set of business behaviors based on a chosen set of criteria. Functions are usually close-coupled to/with organizational units.
Process	A process represents a sequence of activities that together achieve a specified outcome, can be decomposed into sub-processes, and can show operation of a business capability or service.
Business service	Supports the business by encapsulating a unique element of business behavior; a service offered external to the organization may be supported by business services.
Organization unit	A self-contained unit of resources with goals, objectives, and measures. Organization units may include external parties and business partner organizations.

(continued)

Table 5-2. *(continued)*

Entity	Description
Actor	A person, organization, or system that has a role that initiates or interacts with activities. Actors may be internal or external to an organization.
Role	The usual or expected behavior of an actor or the part somebody or something plays in a particular process or event. An actor may have a number of roles.
Product	An outcome generated by the business to be offered to customers. Products include materials and/or services.
Contract	An agreement between a consumer and a provider that establishes functional and non-functional parameters for interaction. This applies to all types of service interactions within the metamodel.
Business information	Represents a concept and its semantics used within the business.
Control	A decision-making step with accompanying decision logic used to determine execution approach for a process or to ensure that a process complies with governance criteria.
Event	An organizational state change that triggers processing events. It may originate from inside or outside the organization and may be resolved inside or outside the organization.
Service quality	A configuration of non-functional requirements or attributes that may be assigned to a business, application, or technology service.

The entities listed in Table 5-2 are further specified in the following paragraphs:

Driver: The condition that motivates the organization to define its goals. This condition can be internal or external.

A *driver* refers to any influence, factor, or condition that pushes or drives the organization to make decisions or take actions that affect its business activities and strategic direction. Drivers are often the reasons behind changes or new initiatives within an organization. They could arise from external environments such as *market share*, *legislation*, or *technological advancements* or from internal conditions like *leadership changes* or *operational inefficiencies*.

Drivers can also be stakeholder related. In this case, they are called *concerns*. In the context of Enterprise Architecture, identifying and analyzing drivers helps architects understand the underlying reasons for change and shape architecture decisions accordingly. Figure 5-6 demonstrates the use of a driver in relation to goals and objectives.

Goal: A high-level statement of intent or direction for an organization, typically used to measure success of an organization.

A *goal* can be anything a stakeholder wants, such as a state of affairs or a value produced. Common examples of goals are *increase profits*, *reduce help desk wait times*, or *implement new regulatory requirements*. Goals are usually expressed in qualitative terms, using words such as *increase*, *improve*, or *easier*. Figure 5-6 shows an example of a goal in the context of the fictional company mentioned in Section 5.1.1.

Objective: An organizational aim that is declared in a SMART² way.

Objectives further detail goals, as they express the measurable aims of the organization with regard to the goals they are related to. For instance, the goal *implement new regulatory requirements* could be related to the objective *make adjustments to comply with legislation demands by Q3 2028* (see Figure 5-6).

²Specific, Measurable, Achievable, Realistic, and Timebound (SMART).

Measure: An indicator or factor that can be tracked, usually on an ongoing basis, to determine success or alignment with goals and objectives.

A *measure* is an entity related to managing and assessing Enterprise Architecture performance. It is used to evaluate how effectively an organization is achieving its goals and objectives through architecture initiatives and projects. A measure is a quantitative indicator used to track progress toward achieving a specific goal or objective. It provides evidence-based insights into the performance and success of an architectural element, initiative, or strategy.

There are three types of measures:

- **Key Performance Indicators (KPIs):** Specific measures tied to business outcomes and critical success factors (CSFs). KPIs are used to monitor progress against strategic goals and are typically outcome-focused.
- **Metrics:** Broader measures that can include both quantitative and qualitative data. These are not always directly tied to outcomes but can measure process efficiency, resource usage, and other operational aspects.
- **Target Values:** Measures often have associated target values that indicate the desired performance level. These target values provide a reference point for evaluating current performance against expectations.

Using Figure 5-6 as an example, a *measurable* indicator could be the *Q3 Legal Department Report*. This report should indicate whether or not the requirements of the new legislation defined in the objective have actually been met, based on certain metrics or target values. Figure 5-6 illustrates the relationship of the measure to entities such as goal and objective.

Course of Action: Direction and focus provided by strategic goals and objectives, often to deliver the value proposition characterized in the business model. A course of action represents what an organization has decided to do.

A *course of action* is an entity that represents an approach or plan to achieve specific objectives or solve a problem. It defines a coordinated set of activities, decisions, and policies that an organization can follow to achieve desired outcomes.

Courses of action can be divided into *strategies* and *tactics*. It is not possible to make a sharp distinction between the two, but *strategies* are typically long-term and fairly broad in scope, while *tactics* tend to be shorter-term and narrower in scope. An example of a (tactical) course of action is shown in Figure 5-6.

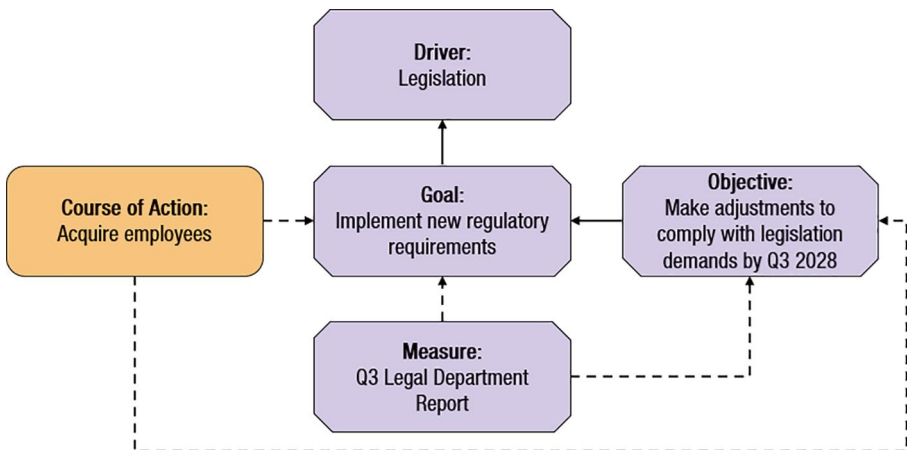


Figure 5-6. Relationship between a driver, goal, objective, measure, and course of action

Figure 5-6 shows that the aforementioned fictional organization (see Section 5.1.1), faced with soon-to-be-enacted legislation, identified legislation as a key driver. In order to comply with the upcoming legislation, the organization has set a goal to implement measures. These

measures must be implemented by the third quarter of the year. One measure to monitor implementation is the preparation of a report by the legal department. This report will indicate whether the requirements have been met. Once the measures have been implemented, a course of action will be set in order to acquire the staff needed to implement them.

Business Capability: A particular ability that a business may possess or exchange to achieve a particular purpose.

The *business capability*, in relation to the entity *capability*, is explained in Section 5.1.1.

Value Stream: A representation of an end-to-end collection of activities that create an overall result for a customer, stakeholder, or end user.

A *value stream* is a core concept used in Enterprise Architecture to focus on how value is delivered to stakeholders, both internal and external, across an organization. Value streams represent the flow of activities or steps that an organization performs to create and deliver products or services that meet stakeholder needs. These activities are not necessarily linear but collectively contribute to the output that provides value. Each value stream consists of *stages*, which are the high-level phases or categories of activities performed to deliver value. The output or outcome of each stage should directly contribute to delivering value to the stakeholder. Value streams are supported by *business capabilities*, which represent the organization's ability to perform activities that support or enable the value stream.

Value streams and *business processes* may seem similar, but they are defined at different levels of abstraction and serve different purposes. A business process describes the (time-ordered) sequence of behaviors required to produce a result for an individual case, and it may describe alternative paths and decision points. In contrast, a value stream focuses on the overall value-creating behavior from the perspective of the importance, worth, or usefulness of what is produced and is not a

description of time-ordered tasks for individual cases. Value streams reflect an organization's *business model* and value proposition, while business processes reflect its *operating model*.

Let's consider the acquisition of new talent (as already indicated by the course of action in Figure 5-6) as an example of the relationship between value streams and business processes. Figure 5-7 shows a simplified example of the stages of the value stream *Acquire Talent*.

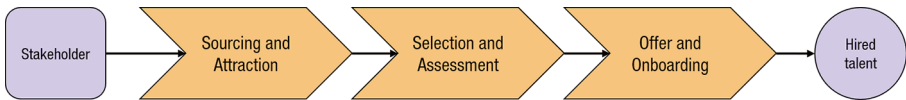


Figure 5-7. Value stream stages of *Acquire Talent*

Each of the value stream stages has a relationship to the underlying process steps being performed:

- Sourcing and Attraction:** Identify and attract potential candidates who fit the organization's needs. During this stage, the process steps that are being followed are creating and posting job descriptions, reaching out to potential candidates, and building a talent pipeline. This stage also includes employer branding efforts to make the organization attractive to prospective candidates.
- Selection and Assessment:** Evaluate and select the most suitable candidates from the pool of applicants. The steps executed by the underlying process are screening resumes, conducting interviews and administering assessments, and narrowing down the candidate pool to those who best meet the job requirements and organizational fit.

- **Offer and Onboarding:** Finalize the hiring process and smoothly integrate the new hire into the organization. The business process steps that support this value stream stage are extending job offers, negotiating terms, and onboarding the new hire. This stage ensures that the candidate transitions effectively into their new role and aligns with the organization’s culture and expectations.

Figure 5-8 shows the addition of process steps to the value stream stages.

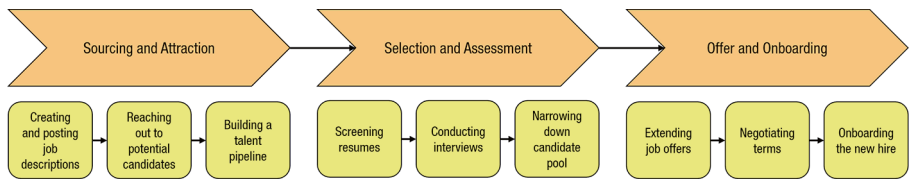


Figure 5-8. *Value stream supplemented by business processes*

The value created by moving through the value stream is *hired talent* (something that is not tangible). A business process describes the steps taken at each stage of the value stream. More specifically, they describe the activities that take place before the acquisition is actually completed.

The end product of the described process steps is a signed employee agreement with the new hire. This is a product that, when printed, can actually be grasped. As such, it becomes a tangible product. Value streams never deliver a tangible end product. They deliver *value*.

A key principle of value streams is that value is always defined from the perspective of the stakeholder – the customer, end user, or recipient of the product, service, or deliverable produced by the work. The value obtained is in the eye of the beholder [14].

An acquired talent (which is the value produced by the value stream) is not tangible (the person is, of course, but the fact that they were hired is not).

Function: A set of business behaviors based on a chosen set of criteria. Functions are usually close-coupled to or with organizational units.

The entity *function* refers to a fundamental building block within the architecture framework. Specifically, it is a component that represents a distinct, cohesive set of activities or services that contribute to achieving a particular outcome or purpose within the organization. A function is defined by its purpose and the responsibilities it holds. It is meant to achieve specific goals or deliver certain outcomes that align with the organization's strategic objectives. Functions are composed of various components such as processes, procedures, and interactions. These components work together to perform the function's role within the broader architecture.

Put more simply, a function is a logical *grouping of activities* that together support the overall operation of a business. Business functions often reflect the organizational structure. However, a function is not tied directly to departments but to the activities themselves.

Process: A process represents a sequence of activities that together achieve a specified outcome, can be decomposed into sub-processes, and can show operation of a business capability or service.

A *process* refers to a set of related, sequential activities or tasks designed to achieve a specific goal or output within the architecture development framework. The TOGAF Standard uses processes to structure and guide the execution of its methodologies and practices, ensuring that activities are carried out systematically and effectively.

Figure 5-8 shows three series of activities associated with three value stream stages. Each set of activities represents the process stages of the corresponding parent processes. The first set of activities is part of the parent process called Acquiring Talent. The second set of activities is part

of the Evaluating Candidates process. The third and final set of activities is part of the Employee Hiring process. Figure 5-9 shows the series of activities, each with its corresponding parent process.

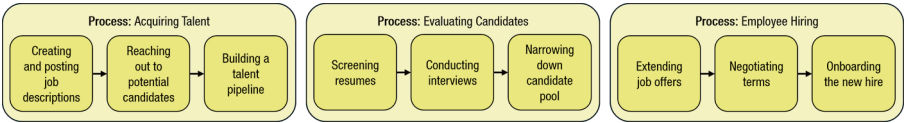


Figure 5-9. *Processes containing series of activities*

The entities *function* and *process* are still sometimes confused. However, the distinction between the two is easy to make. The entity function is broader and more abstract than the entity process. Figure 5-10 illustrates this. Processes define *how things get done step by step*, while functions define *what activities are necessary* to support the organization’s goals.

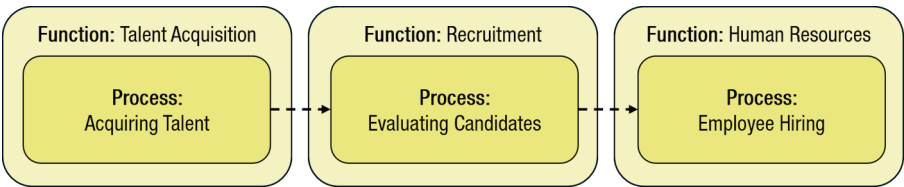


Figure 5-10. *Functions containing corresponding processes*

Organization Unit: A self-contained unit of resources with goals, objectives, and measures. *Organization units* may include external parties and business partner organizations.

Beyond the formal definition, an organization unit can quite easily be compared to a department within an organization. From this perspective, the similarity between an organization unit and a business actor seems quite logical. The TOGAF Standard follows this logic. However, there is a second meaning for this entity. When considering goals, objectives, and measures, the element that comes closest to the way the framework

envision the organization unit is the *stakeholder*. A stakeholder is a role of an individual, team, or organization (or classes thereof) that represents their interests in the outcome of the architecture. Therefore, from a TOGAF perspective, an organization unit is best represented as a stakeholder when considering the relationships to strategic elements such as goals, objectives, and measures.

Actor: A person, organization, or system that has a role that initiates or interacts with activities. Actors may be internal or external to an organization.

The concept of the *actor* describes an entity that interacts with or is affected by an Enterprise Architecture. This interaction can be direct or indirect, and actors are typically external to the architecture itself.

An important distinction that the TOGAF Standard makes here between the entity *actor* and the *organization unit* is that the entity actor can represent an entire organization, but not a department within it. This representation is exclusively for the entity organization unit. Actors therefore usually represent individuals within an organization or external parties. Figure 5-11 illustrates the use of and differences between actors and roles within an organization unit.

Role: The usual or expected behavior of an actor or the part somebody or something plays in a particular process or event. An actor may have a number of roles.

A *role* is defined as a key concept that describes a set of responsibilities, behaviors, and expectations associated with a position or function within an organization or enterprise. The role is typically linked to the tasks and duties that a person or team performs in relation to specific activities or processes. Roles can be assigned to individuals or teams based on their expertise, authority, and function in the organization. A role outlines specific activities, decisions, and accountabilities, which can span across different domains such as business, data, application, and technology.



Figure 5-11. *Actors within an organization unit with an assigned role*

Figure 5-11 shows an organizational unit called Human Resources (HR). The unit consists of two people (John H. Smith and Nina Hunter) who both have the role of HR Business Partner.

Product: An outcome generated by the business to be offered to customers. Products include materials and/or services (accompanied by a contract) and are often the end result of a customer-centric process.

A *product* generally refers to an outcome of a process and should always be of value to a customer. Consider, for example, the process of hiring a new employee. At the end of the process, the *employment agreement* is the *product* offered to the *customer*, in this case the employee. The *terms of employment* are set out in the accompanying *contract*.

In some cases, a product can also be compared to a *business object*, which is a concept that is used within a particular business domain. Figure 5-12 illustrates the use of a product, business objects, and a contract.

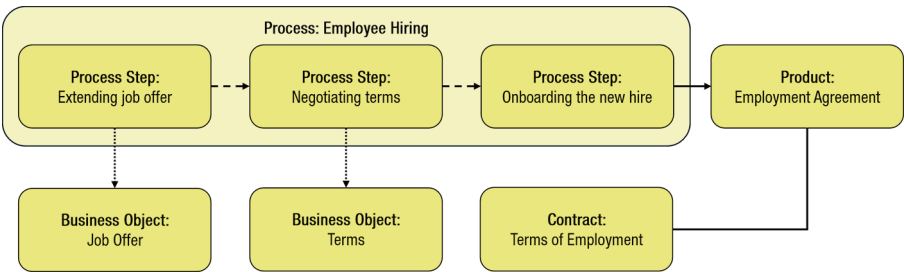


Figure 5-12. *A process creating an end product accompanied by a contract*

Contract: An agreement between a consumer and a provider that establishes functional and non-functional parameters for interaction. This applies to all types of service interactions within the metamodel.

This is a formal agreement that outlines the expectations, responsibilities, and obligations between parties involved in an architectural endeavor. The concept of a *contract* is crucial for defining clear relationships and ensuring alignment between different stakeholders within an architecture.

Contracts in the light of the TOGAF Standard are used to formalize agreements (often related to architecture development and implementation). They help ensure that all parties involved have a mutual understanding of their roles, responsibilities, and deliverables. An example of the entity contract is provided in Figure 5-12.

Business Information: Represents a concept and its semantics used within the business.

The entity *business information* is part of the broader framework for Enterprise Architecture and plays a crucial role in aligning business processes with IT infrastructure. Business information within the context of the TOGAF Standard is concerned with the data and information necessary for business operations and decision-making. It helps to create a clear understanding of what information is critical and of value to the organization, how it should be managed, and how it contributes to the overall business strategy.

Although the definition that the TOGAF Standard gives to this entity is very brief, there is more to be said about it. The beauty of this entity is that it provides a direct link to the information concept from the Business Architecture Body of Knowledge (BIZBOK) Guide. Where the Information Architecture domain disappeared from the TOGAF Standard after version 8 (2002), there now seems to be a movement that could indicate the return of the Information Architecture domain. Perhaps future versions of the TOGAF Standard will confirm this observation.

The BIZBOK Guide defines an information concept as follows:

An information concept is comprehensive in terms of representing objects across a business ecosystem, but may or may not be defined in a data model or database.

The information concept realizes or makes explicit a business object, with the object serving as the basis for the information concept name [5].

In the BIZBOK Guide, the term *information concept* refers to a high-level abstraction that represents a fundamental piece of information that a business uses, manages, and interacts with. Information concepts are not specific data points but rather a representation of the information a business uses to support its processes, products, services, and strategies. Information concepts provide clarity on how information is used across the business.

An information concept is *a key piece of business information*, often derived from business objects. It's not the raw data but a *conceptual model of data* that is meaningful to the business, such as a *strategy* and *vision* or *human resource* and *competency*. These concepts are the foundation for structuring and understanding the information needs of the business, independent of the specific systems or technologies in use.

It's an abstract representation, which means it is not concerned with technical implementation (e.g., databases) but with how the business views and categorizes the information.

Information concepts are relevant in the context of the business and its architecture, helping to ensure alignment between business processes, strategies, and the information needed to support them. They often span multiple domains or functions within an organization, linking processes, capabilities, and value streams.

According to the TOGAF Standard, a data entity (see Section 5.1.3) corresponds to the information concept from the BIZBOK Guide.

The BIZBOK Guide describes Business Architecture (including the information concept, but also information mapping) much more extensively than is done here. For a complete description, it is recommended to consult the BIZBOK Guide.

Control: A decision-making step with accompanying decision logic used to determine an execution approach for a process or to ensure that a process complies with governance criteria.

The entity *control* refers to the mechanisms and processes used to ensure that an organization's architecture aligns with its strategic goals and operates effectively. It encompasses the methods and practices employed to manage, monitor, and guide the implementation of the architecture.

Within the TOGAF Standard, the entity *control* is typically used for the following situations:

- **Governance:** Establishing structures and processes to guide and oversee the development and implementation of the Enterprise Architecture. This includes defining roles, responsibilities, and decision-making processes.
- **Compliance:** Ensuring that the architecture and its implementation adhere to relevant regulations, standards, and policies. This involves regular checks and audits to verify compliance.
- **Monitoring and Reporting:** Implementing mechanisms to track the performance and progress of architectural initiatives. This includes setting up

metrics, monitoring systems, and reporting structures to provide visibility into the effectiveness of the architecture.

- **Risk Management:** Identifying, assessing, and mitigating risks associated with the architecture. This involves understanding potential threats and vulnerabilities and implementing strategies to address them.
- **Change Management:** Managing changes to the architecture in a controlled and systematic way. This includes assessing the impact of changes, coordinating updates, and ensuring that changes align with the overall architectural vision and strategy.

Event: An organizational state change that triggers processing events. It may originate from inside or outside the organization and may be resolved inside or outside the organization.

An *event* is something that has a notable impact or significance, whether it's a change in market conditions, a new regulatory requirement, or something as small as hiring new employees. Events often represent a shift from one state to another, such as candidates being selected after the initial evaluation. Events can be triggered by external factors (new laws, competitive actions) or internal factors (process changes, project milestones). Organizations need to be able to respond to events effectively. This involves understanding the potential impact on the architecture and making necessary adjustments to align with the new conditions.

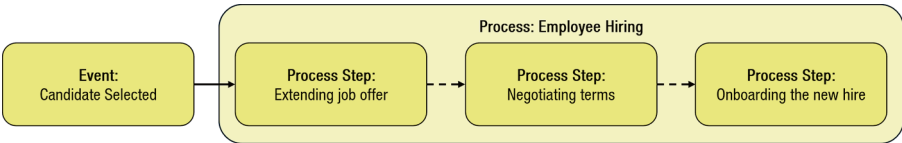


Figure 5-13. *Event triggering a process*

Figure 5-13 shows the process Employee Hiring being triggered by the event of a candidate that is selected.

Service Quality: A configuration of non-functional requirements or attributes that may be assigned to a business, application, or technology service.

The entity *service quality* refers to the measure of how well a service meets or exceeds the expectations of its stakeholders. It encompasses various attributes such as reliability, responsiveness, and user satisfaction. Within the TOGAF Standard, this concept is tied to the broader framework of *IT service management*, which aims to ensure that IT services deliver value to the business and meet defined service levels. The TOGAF Standard emphasizes that service quality should be considered when developing and refining architectural views, particularly in the Technology Architecture domain, with regard to Solution Architecture. The quality of services provided by IT systems and applications is crucial to the overall success of the Enterprise Architecture.

Business Service: Supports the business by encapsulating a unique element of business behavior; a service offered external to the organization may be supported by business services.

A *business service* entity represents a logical, self-contained unit of business functionality that is valuable to the organization or its customers. Business services encapsulate a set of related functions or activities that together support a particular business process or goal. A business service can be provided either to external stakeholders, such as customers or partners, or to internal stakeholders, such as other business units. The consumers of the service typically do not need to know the internal details of how the service is delivered.

The business service appears in the Enterprise Metamodel as the link or layer between the Business Architecture and the Data, Application, and Technology Architectures. This entity is the interface between the more operational architectures on the one hand and the strategic and tactical architectures on the other.



Further Reading

To learn more about the differences and similarities between value streams and business processes, read the “Business Streams And Value Processes” blog:

<https://eawheel.com/blog/2023/05/business-streams-and-value-processes/>

An interesting video on the topic of Business Architecture and value streams can be at the following URL:

<https://m.youtube.com/watch?v=W3yHCffXDM0&t=314s>

See the BIZBOK Guide for more information on Business Architecture (including information concepts and information mapping):

<https://www.businessarchitectureguild.org/page/BIZBOK>

Additional information about information mapping can also be found in the TOGAF Series Guide “Information Mapping”, document number G190.

5.1.3. Data Architecture

The previous section ended with the business service entity as the connecting concept between the Business Architecture on the one hand and the more substantive and operational architectures such as Data, Application, and Technology on the other. This section describes the entities used within the Data Architecture area of the Enterprise Metamodel.

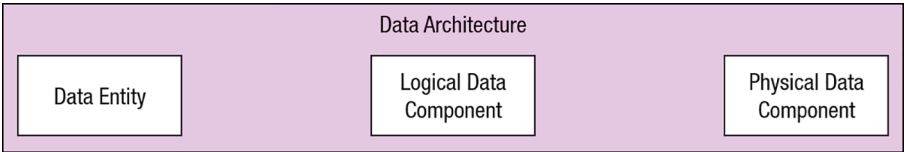


Figure 5-14. *Data Architecture Entities of the Enterprise Metamodel*

The first of the three architectures located at the bottom of the Content Framework is called Data Architecture. Like Application Architecture and Technology Architecture, it contains three entities (see Table 5-3).

Table 5-3. *Data Architecture Entities*

Entity	Description
Data entity	Represents data that is recognized by the business as a distinct concept.
Logical data component	A data structure composed of logically related data entities.
Physical data component	A data structure that realizes related logical data components represented in the format or schema required by a particular technology.

The Data Architecture section of the Enterprise Metamodel is directly related to the business service entity from the Business Architecture section.

Data Entity: Represents data that is recognized by the business as a distinct concept.

Data entities are usually represented in models such as entity-relationship diagrams (ERDs) or class diagrams. These representations include relationships between different entities, constraints, and attributes that further define the nature of the data stored. Data entities are closely tied to the Business Architecture domain. They help to define Business Information Models that guide how data is structured, stored, and used within an organization, ensuring alignment with the business goals.

The data entity has three relationships with entities in the Business Architecture section. First, a data entity is accessed and updated by a business service. Second, it is consumed by an actor, and finally, the data entity realizes a business information entity (see Section 5.1.2).

This realization relationship allows a data entity to exchange data with the business information concept so that it can be transformed into information of value to the organization. Finally, a business information entity helps to create a clear understanding of what information is critical and of value to the organization, how it should be managed, and how it contributes to the overall business strategy.

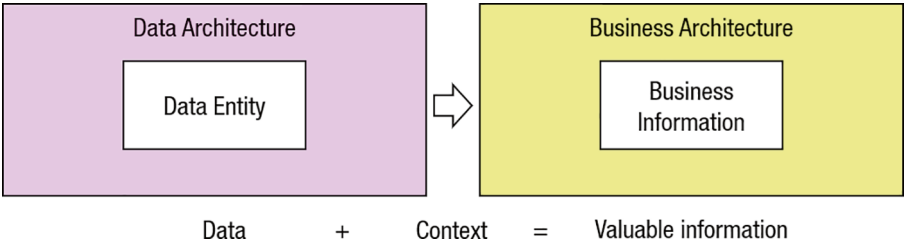


Figure 5-15. Relationship between the data entity and business information

By associating a data entity with a business information entity, it is possible to turn data into valuable information.

For example, consider the numbers 22, 23, 22, 25, and 21. At first glance, these numbers do not mean much. It is only when context is added, such as indicating that this is the expected temperature for the next five days, that the data set gains value and becomes information. This is illustrated in Figure 5-15.

Logical Data Component: A data structure composed of logically related data entities.

The purpose of defining *logical data components* is to represent the abstract, technology-agnostic structures of the data needed by the business, independent of any specific technology or implementation. As the name of the entity already suggests, they represent the *logical structure of data* without regard to how the data is physically stored or implemented. The focus is on *what the data is* and how it relates to business operations

rather than how or where it is stored. Logical data component entities are *abstract* representations, meaning they do not concern themselves with databases, file systems, or other technologies.

Physical Data Component: A data structure that realizes related logical data components represented in the format or schema required by a particular technology.

A *physical data component* represents the *tangible* implementations of data systems, including the infrastructure (hardware and software) needed to store, access, and manipulate data. It could refer to databases, file servers, data warehouses, cloud storage solutions, and similar technologies.

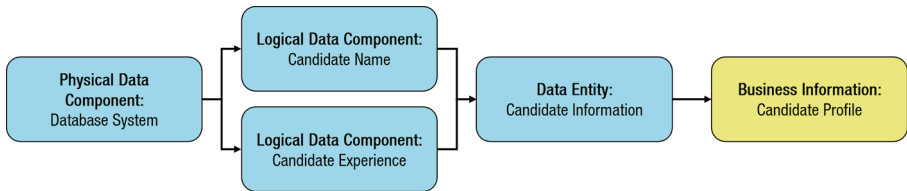


Figure 5-16. *Physical and logical data components in relation to a data entity and business information*

Physical data components are implementations of logical data components. While a logical data component focuses on abstract data structures and conceptual representations of data (such as database schema or data models), the physical component is the actual infrastructure used to implement these structures. For example, a logical component might describe the type of data being stored (e.g., candidate information), while the physical component specifies which database system stores that data. Figure 5-16 illustrates the relationship between physical and logical data components on the one hand and data entities and business information on the other.

5.1.4. Application Architecture

The business service entity serves as the connecting concept between the Business Architecture on the one hand and architectures such as Data, Application, and Technology on the other. This section describes the entities used within the Application Architecture area of the Enterprise Metamodel.

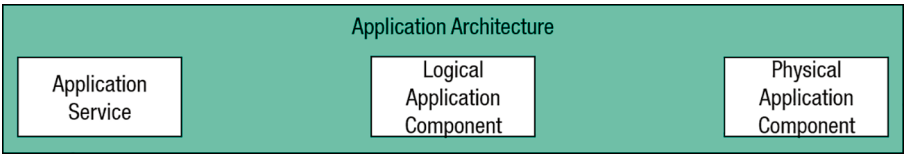


Figure 5-17. *Application Architecture Entities of the Enterprise Metamodel*

The second of the three architectures located at the bottom of the Content Framework is called Application Architecture. It contains three entities (see Table 5-4).

Table 5-4. *Application Architecture Entities*

Entity	Description
Application service	The automated elements of a business service. An application service may deliver or support part or all of one or more business services.
Logical application component	An encapsulation of application functionality that is definable by services offered and data maintained, independently of implementation and technology.
Physical application component	A realization of logical application functionality using components of functionality in applications that may be hired, procured, or built.

Application Service: The automated elements of a business service. An application service may deliver or support part or all of one or more business services.

An *application service* entity delivers specific functionality that an organization needs. It typically supports the business processes and operations by offering automated or semi-automated functions.

Logical Application Component: An encapsulation of application functionality that is definable by services offered and data maintained, independently of implementation and technology.

The *logical application component* entity refers to an abstract representation of software that performs specific *functions* or *services* in an architecture. This component encapsulates the behavior and responsibilities of a system without getting into the physical implementation details, which allows architects to focus on *what* the system should do rather than *how* it is realized.

The logical application component entity represents the *abstract* or *conceptual* view of an application. It describes its functionality, interfaces, and behaviors without reference to technology specifics.

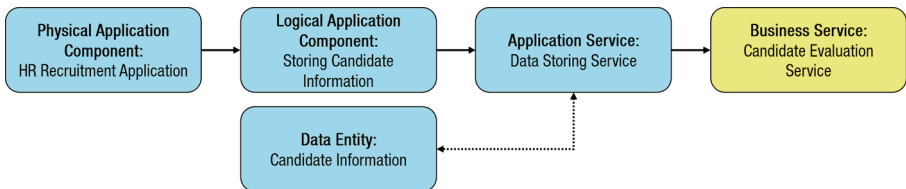


Figure 5-18. Relationship between physical and logical application components and application and business services

The use of a logical application component and an application service is depicted in Figure 5-18. A logical application component entity may represent the system that handles HR recruitment *functionalities* like storing candidate information and tracking interviews. This logical component doesn't specify if the system will be implemented using HR

Recruitment Application A, B, or a custom-built application or platform – that decision happens in the physical design phase. A logical application component focuses on the functionality it provides.

The application service entity is implemented by the logical application component and uses a business service to provide functionality to the organization. The application service may also use one or more data entities. Figure 5-18 illustrates this example scenario.

Physical Application Component: A realization of logical application functionality using components of functionality in applications that may be hired, procured, or built.

The *physical application component* entity is an actual, concrete implementation of an application system or software that runs on a specific technology platform. It refers to the *tangible* aspects of an application, including the installed software, the configurations, and the underlying hardware resources required to operate the application. The physical application component entity describes the implementation-specific details, such as the software packages, configurations, and deployment environments (see Figure 5-18).

5.1.5. Technology Architecture

The business service entity mentioned in the previous two sections also connects the Business Architecture layer to the Technology Architecture layer. This section describes the entities used within the Technology Architecture area of the Enterprise Metamodel.

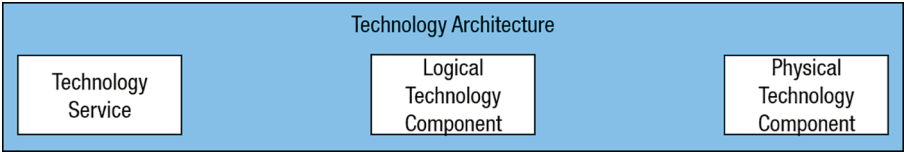


Figure 5-19. *Technology Architecture Entities of the Enterprise Metamodel*

The third of the three architectures located at the bottom of the Content Framework is called Technology Architecture. Like Data Architecture and Application Architecture, it contains only three entities (see Table 5-5).

Table 5-5. *Technology Architecture Entities*

Entity	Description
Technology service	A technical capability required to provide enabling infrastructure that supports the delivery of applications.
Logical technology component	An implementation-independent encapsulation of technology services.
Physical technology component	A realization of logical technology functionality using a particular technology product that may be deployed.

Technology Service: A technical capability required to provide enabling infrastructure that supports the delivery of applications.

Much like the application service entity mentioned in Section 5.1.4, and as the name suggests, the *technology service* provides a service that other entities can use. It represents the technical capabilities provided by the underlying technology infrastructure that support the higher-level business, application, and data services in the organization.

Consider a cloud storage service within an Enterprise Architecture. This cloud-based service provides scalable, secure, and on-demand storage resources to various applications and users across the organization. Think, for example, about storing files, handling data backups, and ensuring data accessibility from different geographical locations. Figure 5-20 demonstrates the way a technology service entity delivers its services to other entities.

Logical Technology Component: An implementation-independent encapsulation of technology services.

The *logical technology component* entity is an abstraction representing a technology-related function or capability that does not include any specific physical implementation details. It focuses on the functionality required rather than the actual physical hardware, software, or network that performs that function.

An example of a logical technology component could be a Database Management System (DBMS). At this level, the logical technology component does not specify whether the DBMS will be MySQL, Oracle, SQL Server, or another product. It simply indicates the need for a database system that can manage, store, and retrieve data in a structured manner.

Consider Figure 5-20. The aforementioned fictitious company's architecture (see Section 5.1.1) needs to manage large amounts of candidate data. The architecture specifies a logical technology component entity called Database Management System (DBMS) as part of the Technology Architecture. The DBMS manages structured candidate data, supports SQL queries, ensures data integrity, provides backup capabilities, etc. It does not, however, specify the specific database product, vendor, or version. Should the organization select MySQL as the database system (the actual solution), the logical component DBMS would then be realized by a specific physical technology component.

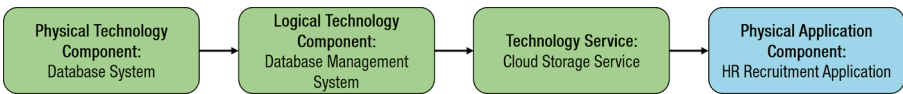


Figure 5-20. Relationship between physical and logical technology components and technology and application services

Physical Technology Component: A realization of logical technology functionality using a particular technology product that may be deployed.

The entity *physical technology component* refers to the actual, tangible parts of an IT system, such as hardware, software, and networking devices, that are deployed in an organization's Technology Architecture. These components physically exist and operate in the real world, enabling the

functioning of the overall system. Figure 5-20 illustrates the coherence and relations between a technology service, a logical technology component, and a physical technology component.



Further Reading

The “TOGAF Content Framework and Enterprise Metamodel” chapter of the TOGAF Standard provides additional information on the form and application of the Content Framework and Enterprise Metamodel.

For more information on the TOGAF Standard’s view of business capabilities, see the “Business Capabilities, Version 2” Series Guide, document number G211.

5.2. Tailoring the Enterprise Metamodel

The Enterprise Metamodel, the Content Framework, and the Architecture Development Method are inextricably linked. The Architecture Development Method is often considered the heart of the TOGAF Standard and plays a very important role in the triangle just described. The Architecture Development Method allows architects to create an *organization-specific* version of the Enterprise Metamodel. The necessary steps for this take place during the Preliminary Phase (refer to Chapter 12, Section 12.3, for additional information).

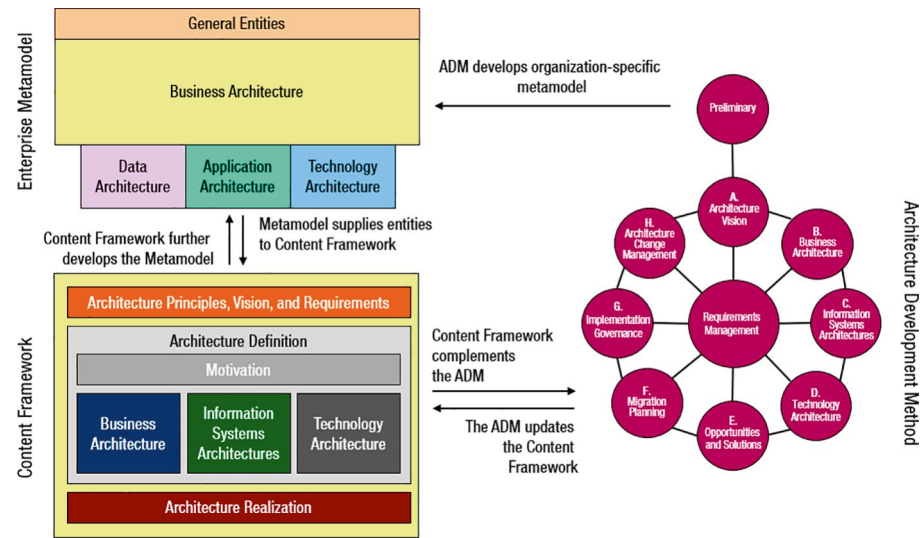


Figure 5-21. Relation between ADM, Enterprise Metamodel, and Content Framework

Please note: The arrows in Figure 5-21 do not represent a direct connection between a model on the one hand and a specific phase of the Architecture Development Method on the other. All arrows point *from one model to another*, not to specific parts of a model.

Once the Enterprise Metamodel - of which the TOGAF Standard describes a basic model that must be adapted to the situation, wishes, and requirements of the organization - is formed, this model serves as input for the Content Framework. The organization-specific Enterprise Metamodel contains and describes all the entities that exist and are needed within the organization. These entities are made available to the Content Framework.

The Content Framework describes and structures architecture within an organization. It consists of a collection of models that represent the architecture, making use of the entities from the Enterprise Metamodel. The Content Framework uses the described entities to create architecture deliverables and artifacts. It also enables the evolution of the Enterprise Metamodel.

The Enterprise Metamodel serves as a handle or starting point, and it is strongly recommended that the metamodel be adapted to one's organization. This means that the Enterprise Metamodel allows - even suggests - adding new entities or removing entities that are not relevant to the organization.

Note that the TOGAF Standard does not aim to constrain an enterprise's selection of artifacts [2].

The TOGAF Standard's view of the Enterprise Metamodel is that it can be used as a basis for developing an organization-specific metamodel. This development takes place during the Preliminary Phase when the Enterprise Architecture Capability is being established.

When developing an organization-specific metamodel, architects may choose not to include entities and relationships from the Enterprise Metamodel that are not relevant and/or add additional entities and relationships [2].

The tailorability of the TOGAF Standard is highlighted once again by allowing this flexibility in the application of the Enterprise Metamodel.

Although the TOGAF Standard does not explicitly state it, the organization unit entity in the Enterprise Metamodel is used to refer to stakeholders.

The TOGAF Standard indicates that an organization unit is “*a self-contained unit of resources with goals, objectives, and measures*”. This is consistent with the fact that stakeholders are typically associated with goals and objectives.

However, I would strongly suggest adding the entity stakeholder to the Enterprise Metamodel. This is an essential entity that is used quite often when creating architecture deliverables and artifacts.

ArchiMate, on the other hand, does recognize the stakeholder element and recommends using this element when the context is about goals and objectives and therefore strategic in nature:

“We propose to use the ArchiMate definition of Stakeholder when using the TOGAF definition of Organization Unit with relations to Measure, Goal, or Objective” [15].

The addition of the stakeholder entity to the Enterprise Metamodel contributes to the integration between the TOGAF Standard and the ArchiMate modeling language, also a standard managed by The Open Group.

The first iteration of the Architecture Development Method focuses specifically on creating the Architecture Capability. Part of the capability creation is the development of an initial version of the Enterprise Metamodel. This involves creating an organization-specific variant of the Enterprise Metamodel as presented in the TOGAF Standard. This step is important and deserves to be emphasized again, as many architects tend to overlook it.

It makes perfect sense to add entities such as stakeholder, business collaboration, business interaction, business interface, resource, risk, and value to the organization-specific metamodel. For example, the stakeholder entity is used in the Stakeholder Catalog, and most of the other listed entities appear in the Business Model Diagram (see Chapter 12, Section 12.4.2).

The Enterprise Metamodel is populated with the entities that exist in the organization. If an organization has exceptional entities, they can (and need to) be added to the metamodel. Once the Enterprise Metamodel is customized, it is used as input to the Content Framework. This framework describes all the architecture deliverables (consisting of artifacts and building blocks) that are meaningful to produce during the architecture work. The existing Content Framework in the TOGAF Standard shows a basic model. And again, this base model needs to be tailored according to which entities are described in the organization-specific Enterprise Metamodel.

When the Content Framework is populated with architecture deliverables, they can in turn be made available to the phases from the Architecture Development Method. As you move through the phases, the architecture deliverables are created according to how they are included in the Content Framework. Of course, it is and will remain possible to make adjustments to both the Enterprise Metamodel and the Content Framework as necessary. Architecture constantly evolves, and the concepts used must evolve with it.



Understand the Enterprise Metamodel: Get familiar with the Foundation Enterprise Metamodel provided in the TOGAF Standard. This model outlines standard entities and relationships used in Enterprise Architecture.

Customize the Metamodel: Identify and include any exceptional entities that are unique to the organization. Adding these entities ensures the metamodel accurately represents the organization's architecture.

Integrate the Customized Metamodel: Use the tailored Enterprise Metamodel as input for the Content Framework. This framework defines all architecture deliverables, including artifacts and building blocks, that are relevant during architecture development. Adding or removing architecture deliverables to and from the Content Framework will customize it to the organization's needs.

Align Deliverables: When progressing through the phases of the Architecture Development Method, create architecture deliverables in accordance with the customized Content Framework. This alignment ensures consistency and relevance in the architecture work.

Continuously Evolve: Recognize that architecture is an evolving discipline. Regularly review and update both the Enterprise Metamodel and the Content Framework to reflect changes in the organization and its environment. This ongoing refinement maintains the effectiveness and accuracy of the Enterprise Architecture.

Tailoring the Enterprise Metamodel is an absolute necessity. The foundation model provided in the TOGAF standard is only a starting point. Be sure to spend sufficient time creating an organization-specific model before proceeding with the development of the Content Framework.



Further Reading

For more information on customizing the Enterprise Metamodel, see “The TOGAF® Leader’s Guide to Establishing and Evolving an EA Capability”, Chapter 8, Customization of Architecture Contents and Metamodel, document number G184.

More information on using ArchiMate to support the TOGAF Standard can be found in the document “How to Use the ArchiMate® Modeling Language to Support the TOGAF® Standard”, document number G21E.

5.3. Summary

One of the core components of the TOGAF Standard is the Enterprise Metamodel. This metamodel contains the entities and their relationships as they are likely to occur in practice in most organizations. The Enterprise Metamodel provides the context for the specific artifacts referenced in the descriptions of the Architecture Development Method phases.

The entities from the Enterprise Metamodel form the basis for the Content Framework. Both the Enterprise Metamodel and the Content Framework are kept in sync; the Enterprise Metamodel provides the entities used in the Content Framework, and the Content Framework evolves the Enterprise Metamodel.

The Enterprise Metamodel comprises five interconnected sections:

- General Entities
- Business Architecture
- Data Architecture
- Application Architecture
- Technology Architecture

Each section consists of a set of model entities. These entities are architecture concepts, each with its own definition and explanation.

The *General Entities* are concepts that are so generic in nature that they cannot be captured or categorized in any of the four architecture domains. They can be used across all four domains. The General Entities consist of eight entities.

All 19 architecture concepts that are in the *Business Architecture* environment can be classified as entities in this context. However, the way the TOGAF Standard views certain entities such as driver, goal, objective, and measure is different from the way the ArchiMate modeling language views them. The ArchiMate standard assigns these entities to the *motivation elements*. The TOGAF Standard places all these entities under one heading: Business Architecture. The reason for this is that the TOGAF Standard does not have motivational or strategic elements, but uses the architecture domains (Business, Data, Application, and Technology).

The *Data Architecture* section of the Enterprise Metamodel is directly related to the business service entity from the Business Architecture section. It contains three entities. The same goes for the *Application Architecture* and *Technology Architecture* sections. They contain three entities as well.

All 36 entities have been described in detail in this chapter, along with examples of how to use them in practice. The entities in the Enterprise Metamodel are part of what is called the Foundation Enterprise Metamodel, a standard set of entities to start with. One of the key aspects of using the framework is to tailor it to the needs of the organization. This includes the basic set of entities that are part of the Enterprise Metamodel. Entities can be added to or removed from the base set, depending on the needs of the organization.

The Enterprise Metamodel, the Content Framework, and the Architecture Development Method are inextricably linked. The Architecture Development Method allows architects to create an organization-specific version of the Enterprise Metamodel. The necessary

steps for this take place during the Preliminary Phase. Once the Enterprise Metamodel is formed, it serves as input for the Content Framework. The organization-specific Enterprise Metamodel contains and describes all the entities that exist and are needed within the organization. These entities are made available to the Content Framework.

The steps for creating a tailored Enterprise Metamodel are as follows:

- **Understand the Enterprise Metamodel:** Get familiar with the Foundation Enterprise Metamodel provided in the TOGAF Standard. This model outlines standard entities and relationships used in Enterprise Architecture.
- **Customize the Metamodel:** Identify and include any exceptional entities that are unique to the organization. Adding these entities ensures the metamodel accurately represents the organization's architecture.
- **Integrate the Customized Metamodel:** Use the tailored Enterprise Metamodel as input for the Content Framework. This framework defines all architecture deliverables, including artifacts and building blocks, that are relevant during architecture development. Adding or removing architecture deliverables to and from the Content Framework will customize it to the organization's needs.
- **Align Deliverables:** When progressing through the phases of the Architecture Development Method, create architecture deliverables in accordance with the customized Content Framework. This alignment ensures consistency and relevance in the architecture work.

- **Continuously Evolve:** Recognize that architecture is an evolving discipline. Regularly review and update both the Enterprise Metamodel and the Content Framework to reflect changes in the organization and its environment. This ongoing refinement maintains the effectiveness and accuracy of the Enterprise Architecture.

The Enterprise Metamodel is the foundation of the Enterprise Architecture. It takes time and care to create a customized and organization-specific version of the metamodel. The model serves as input to both the Content Framework and the Architecture Development Method.